Advanced TCP/IP Analysis and Troubleshooting with Wireshark: Lab Report

SARAH MORSHED
sarahmorshed@gmail.com

Objective 1: Diagnose TCP Performance Issues from Packet Captures

The purpose of this objective was to analyse TCP performance during a file download by identifying retransmissions and evaluating whether TCP windowing contributed to packet loss. Wireshark filtering and packet-level analysis were used to investigate the cause of retransmissions.

<u>Step 1: Identifying TCP Retransmissions</u>

Action performed Opened *TCP.pcapng* and applied the filter '*tcp.analysis.retransmission*' to isolate retransmitted packets.

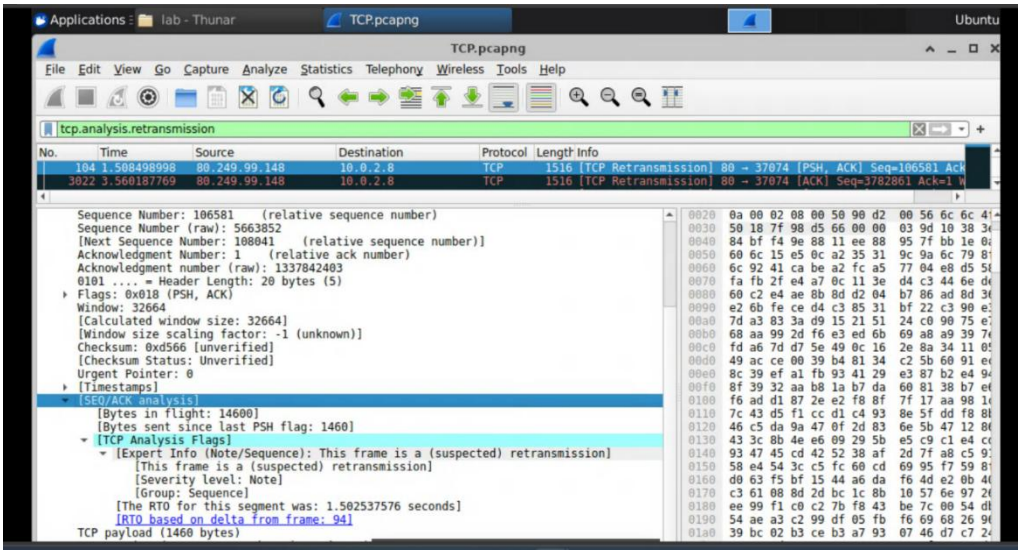

Figure 1.1 *TCP retransmissions identified using tcp.analysis.retransmission filter.*

Observation Several retransmissions were visible between the external and internal hosts, showing that some packets had to be resent because they were not successfully received the first time.

Why This Matters TCP retransmissions usually indicate packet loss which can point to network performance or connectivity problems.

<u>Step 2: Analysing TCP Window Size Before Retransmission</u>

Action Performed Filtered for packet *'frame.number == 4424'* to inspect the ACK packet sent immediately before the retransmission occurred.

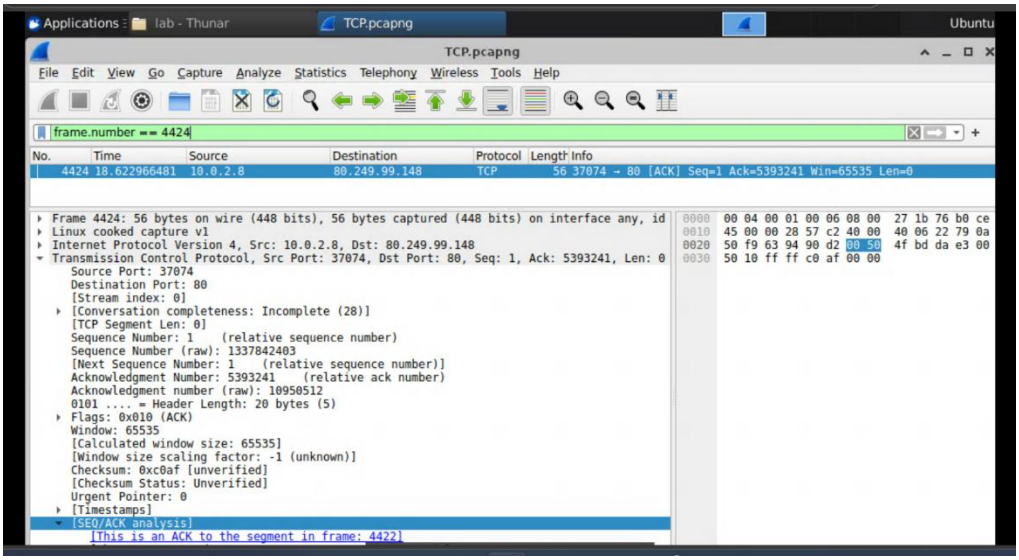

Figure 1.2 *Packet 4424 showing TCP ACK window size prior to retransmission event.*

Observation The packet shows that the client advertised a large TCP receive window, meaning it was able to accept more incoming data from the server.

Why This Matters Since the receiver still had available buffer space, it is unlikely that packet loss was caused by the client being overwhelmed..

<u>Step 3: Analysing Retransmitted Segment Length</u>

Action Performed Filtered for packet 'frame.number == 4429' and reviewed packet details to inspect the size of the retransmitted segment.

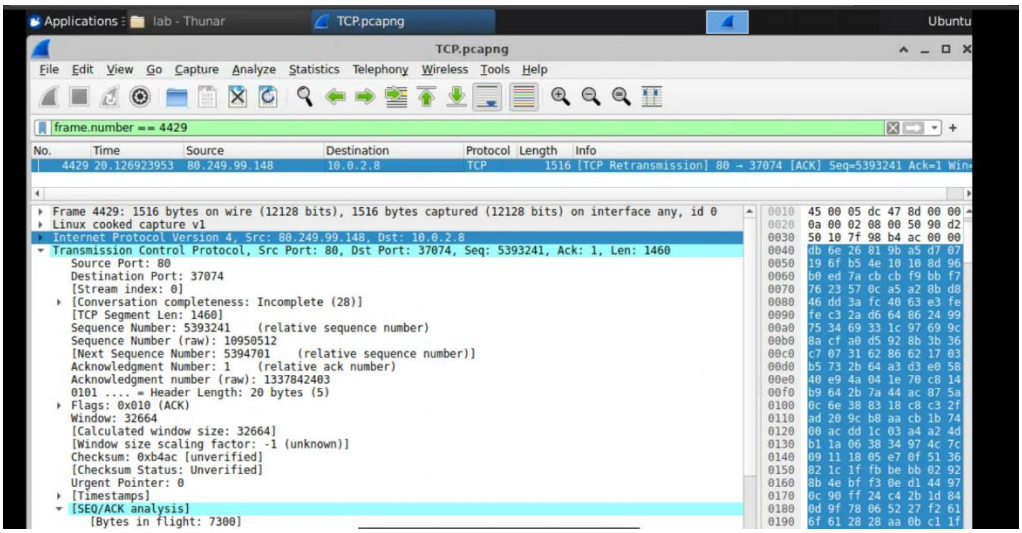

Figure 1.3 *Packet 4429 highlighting retransmitted segment length.*

Observation The retransmitted segment size was 1460 bytes, which fits within the receive window previously advertised by the client.

Why This Matters Because the segment size was within acceptable limits, the packet loss was more likely caused by network or host-related issues rather than buffer overload.

<u>Step 4: Review Packet Sequence Leading to Retransmission</u>

Action Performed Removed filters and sorted packets by frame number to review the packet flow between packets 4424 and 4429.

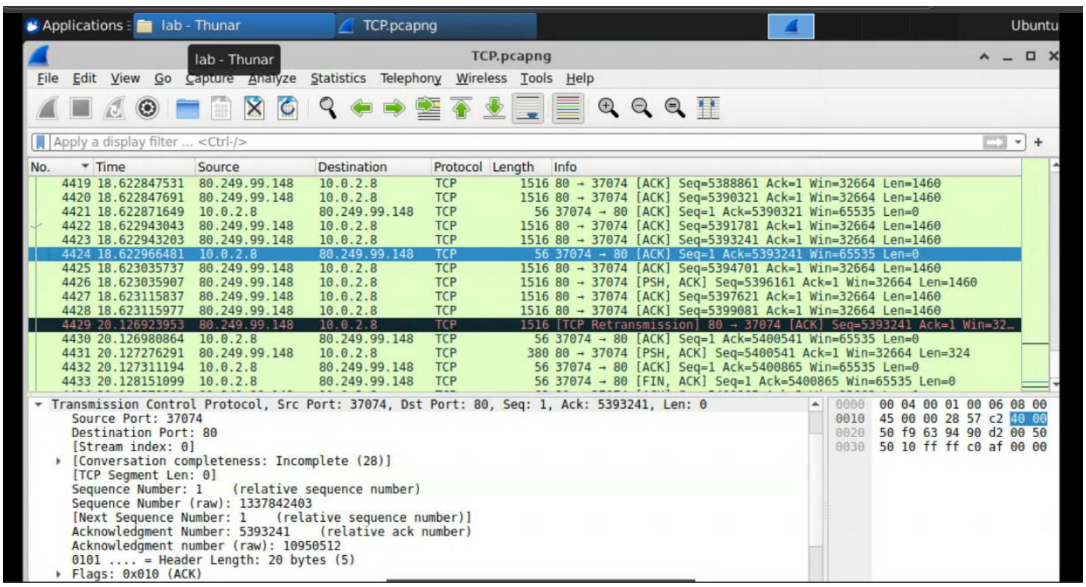

Figure 1.4 *Ordered packet sequence showing traffic flow leading to TCP retransmission.*

Observation Viewing the packets in order made it easier to see the normal flow of traffic leading up to the retransmission.

Why This Matters Reviewing packet order helps show how retransmissions occur within a TCP session and provides useful context when troubleshooting network performance.

Objective 2: Troubleshooting Common Network Protocol Failures

Step 1: Identifying ARP Replies

Action Performed Opened ARP.pcap and applied the filter arp.opcode == 2 to display ARP reply packets.

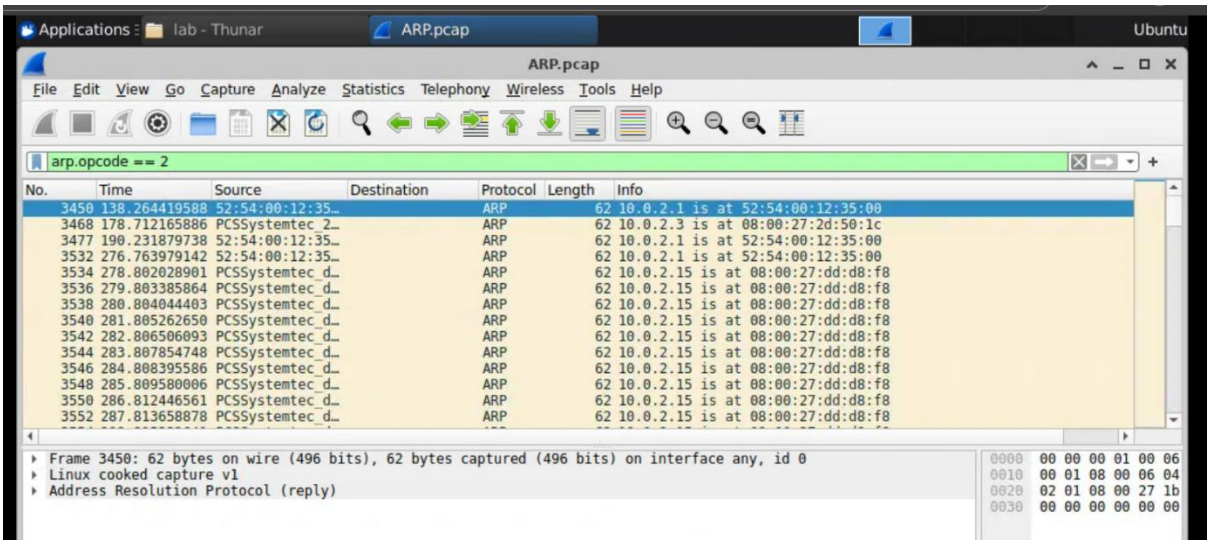

Figure 2.1 Filtered ARP replies.

Observation The filtered results showed ARP replies where devices on the network were advertising IP-to-MAC address mappings.

Why This Matters ARP replies help identify how devices map IP addresses to physical MAC addresses, which is important when looking for abnormal or suspicious network behaviour.

Step 2: Detecting Changes in IP-to-MAC Mapping

Action Performed Reviewed ARP packets and compared entries for the same IP address across multiple packets.

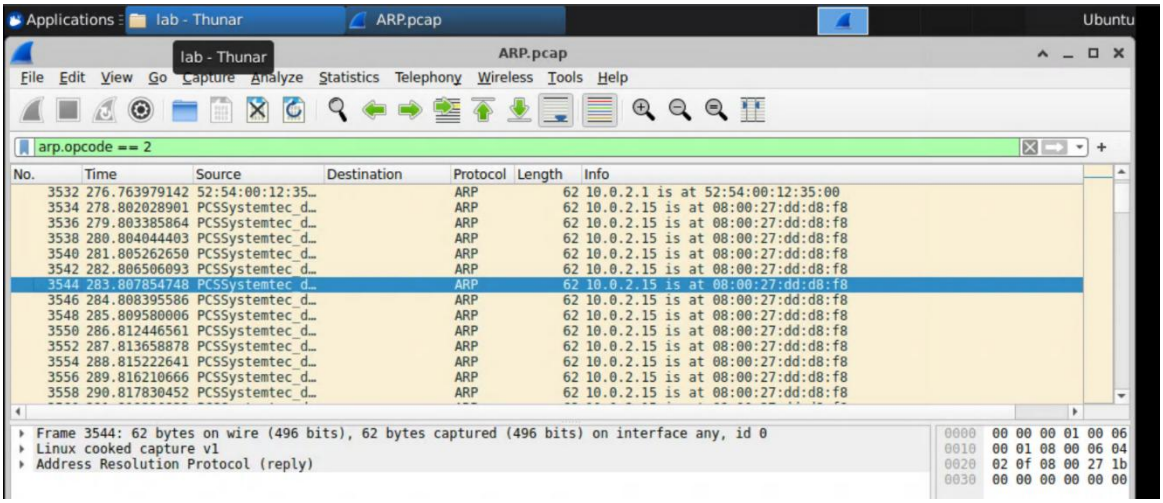

Figure 2.2 Packet 3544 having IP 10.0.2.15 is at MAC address 08:00:27:dd:d8:f8.

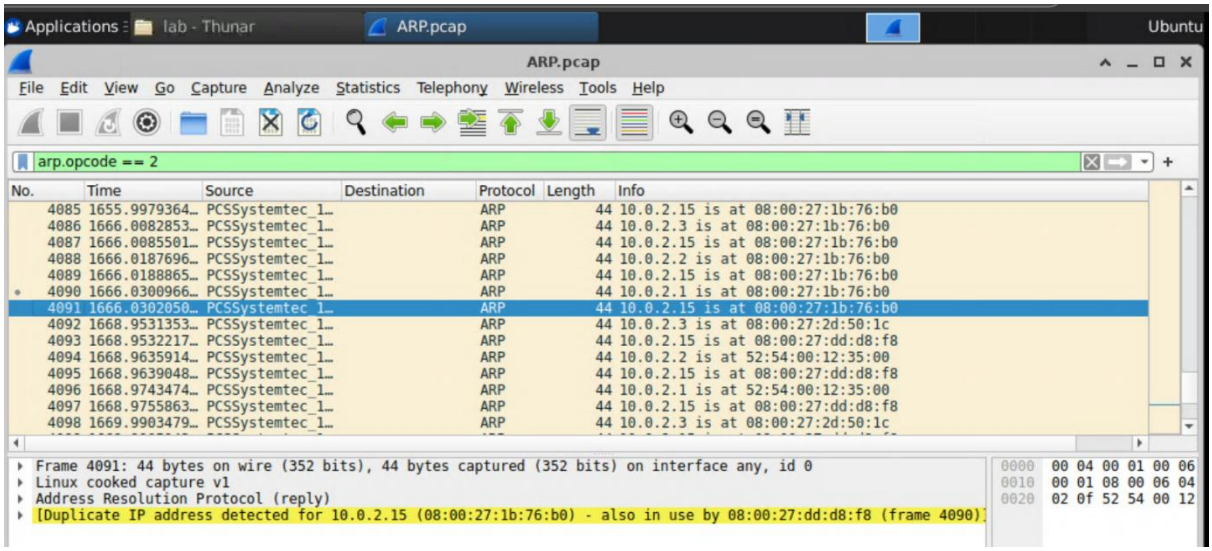

Figure 2.3 Packet 4091 shows IP 10.0.2.15 with different MAC address.

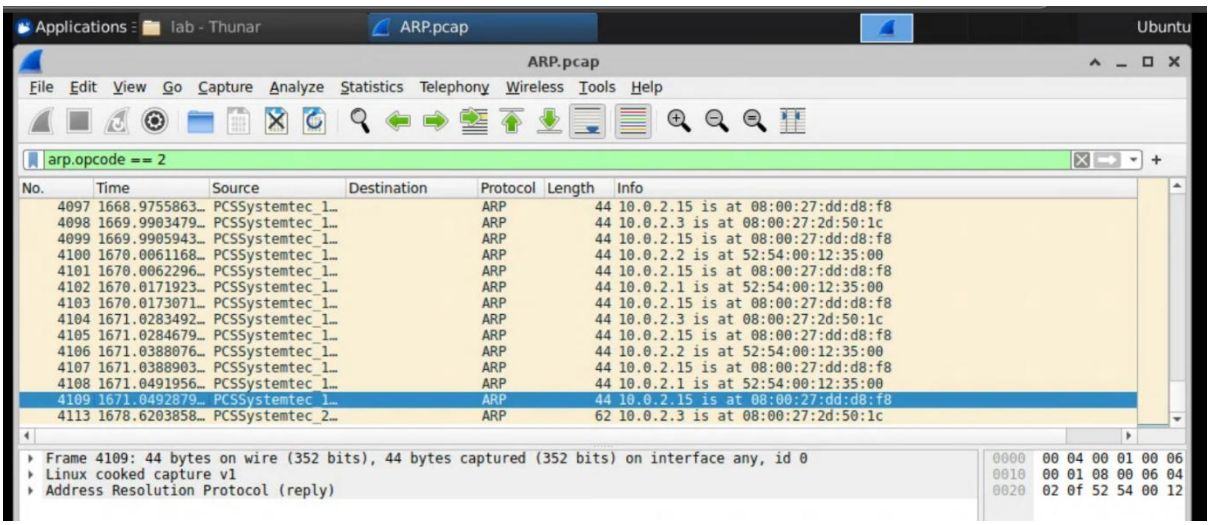

Figure 2.4 Packet 4109 shows IP 10.0.2.15 is mapped back to original MAC address.

Observation The IP address 10.0.2.15 appeared with different MAC addresses at different points in the capture before returning to its original value.

Why This Matters Frequent changes in IP-to-MAC mappings are a common indicator of ARP spoofing where an attacker may try to redirect traffic.

Step 3: Reviewing Wireshark Warning Indicators

Action Performed Selected packet 4013 and reviewed warning messages in Wireshark Expert Information.

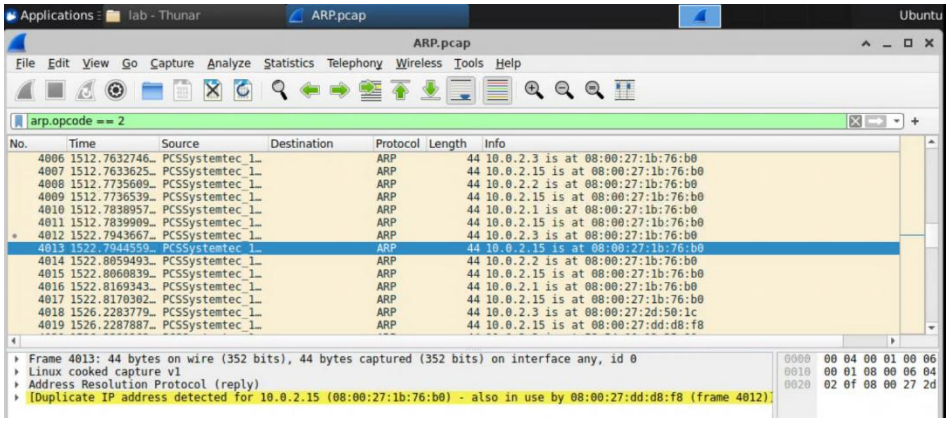

Figure 2.5 Packet 4013 with warning message about Duplicate IP Address detection for 10.0.2.15.

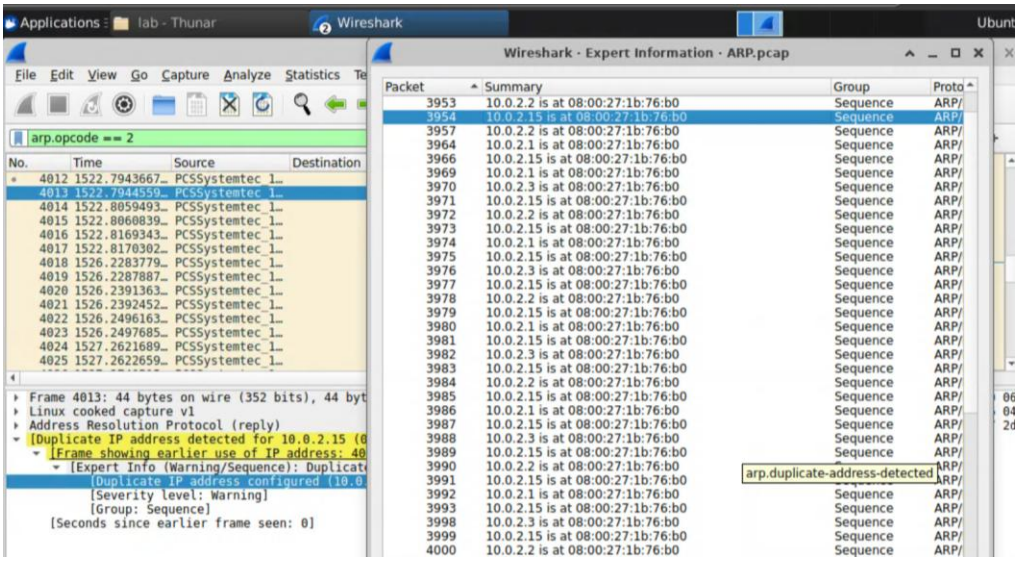

Figure 2.6 Wireshark Expert Information expanded.

Observation Wireshark displayed a warning about a duplicate IP address for 10.0.2.15.

Why This Matters Duplicate IP warnings often occur during ARP spoofing attempts, providing additional evidence that the network mapping may have been manipulated.

Step 4: Investigating DNS Traffic (HTTP Requests)

Action Performed Opened DNS.pcapng and applied the http filter to inspect HTTP requests.

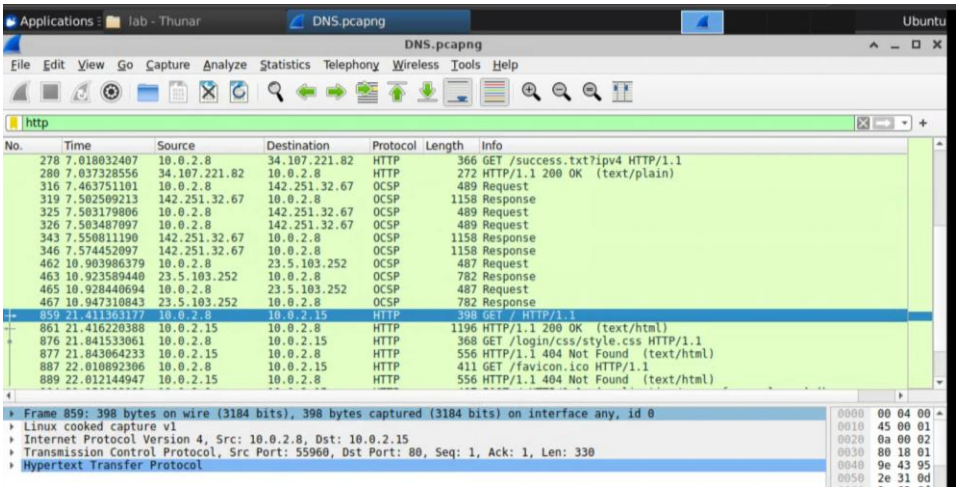

Figure 2.7 HTTP GET request selected for DNS resolution analysis

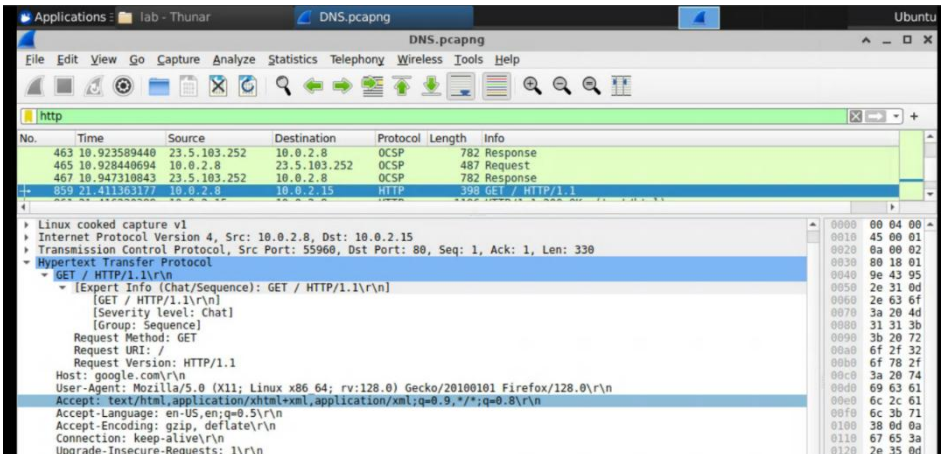

Figure 2.8 HTTP host header analysis showing possible DNS poisoning

Observation An HTTP request showed the host as google.com but the destination IP address was an internal address.

Why This Matters A mismatch between the requested domain and destination IP means possible DNS poisoning or incorrect DNS resolution.

<u>Step 5: Reviewing DNS Responses</u>

Action Performed Applied the filter dns.flags.response == 1 to view DNS response packets and reviewed the associated traffic.

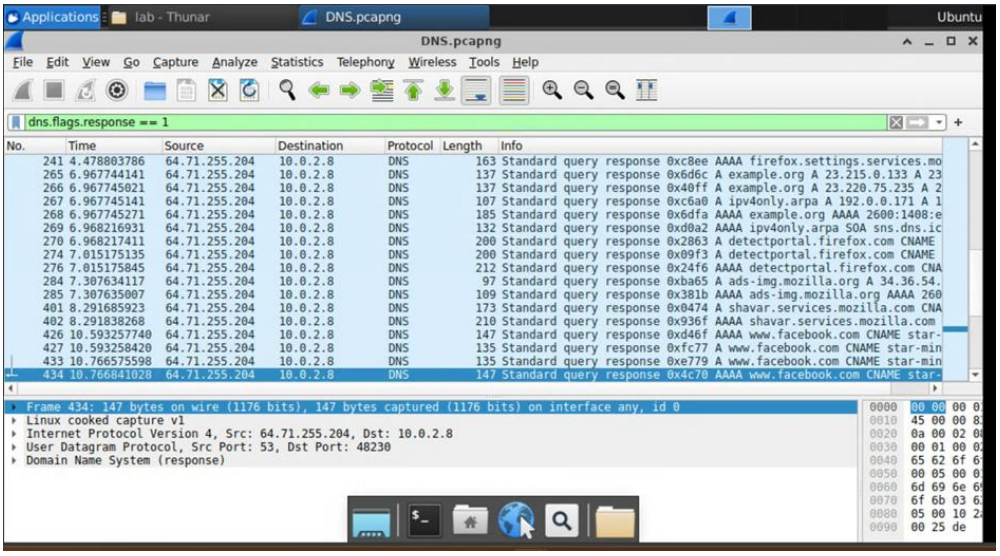

Figure 2.9 DNS response packet showing successful resolution for www.facebook.com used to compare normal DNS behaviour against earlier suspicious traffic.

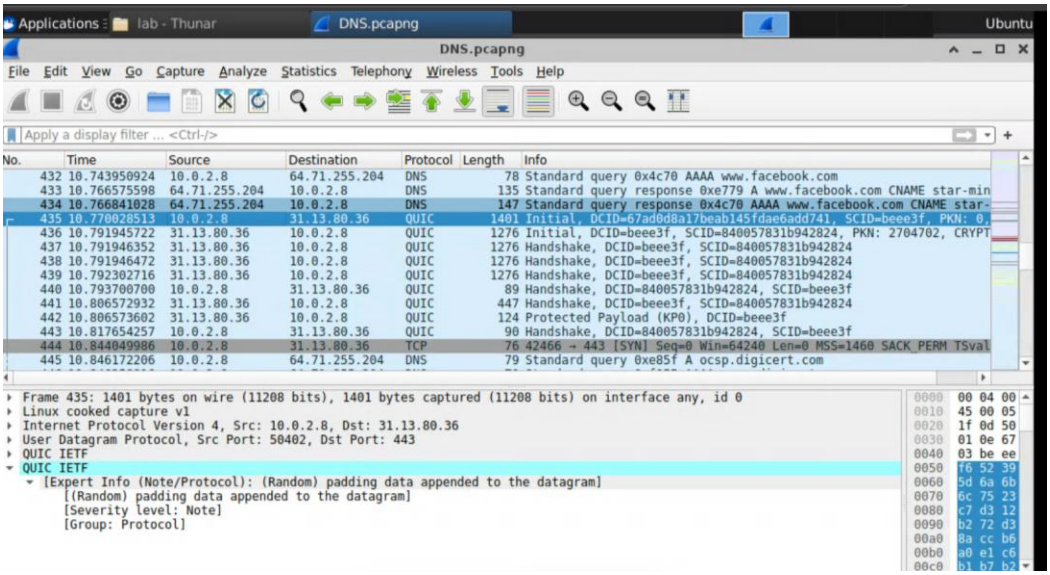

Figure 2.10 Packet 435 showing connection to resolved Facebook server IP (31.13.80.36) following successful DNS resolution.

Observation The DNS response successfully resolved another domain (Facebook) to a public IP address, followed by a normal connection.

Why This Matters Comparing normal DNS resolution with suspicious traffic helps confirm that the issue is specific to certain domains rather than a complete DNS failure.

Objective 3: Utilize Wireshark Features for In-Depth Analysis and Problem Isolation

Step 1: Filtering FTP Traffic

Action Performed Opened FTP.pcapng and applied the filter ftp to isolate FTP-related traffic.

Observation Filtering reduced the capture to FTP packets only, making it easier to focus on the application-layer communication.

Why This Matters Filtering traffic allows analysts to quickly isolate specific protocols during investigations and reduces noise in large packet captures.

Step 2: Following the TCP Stream

Action Performed Right clicked the first FTP packet and selected Follow → TCP Stream to reconstruct the full FTP conversation.

Observation The entire FTP session appeared in one window, showing the communication between client and server in plaintext.

Why This Matters Following TCP streams helps analysts view full conversations and understand application-level activity, especially when protocols are unencrypted.

Step 3: Exporting the FTP Conversation

Action Performed Used the Save As option within the TCP stream window to export the conversation as FTP_Conversation.

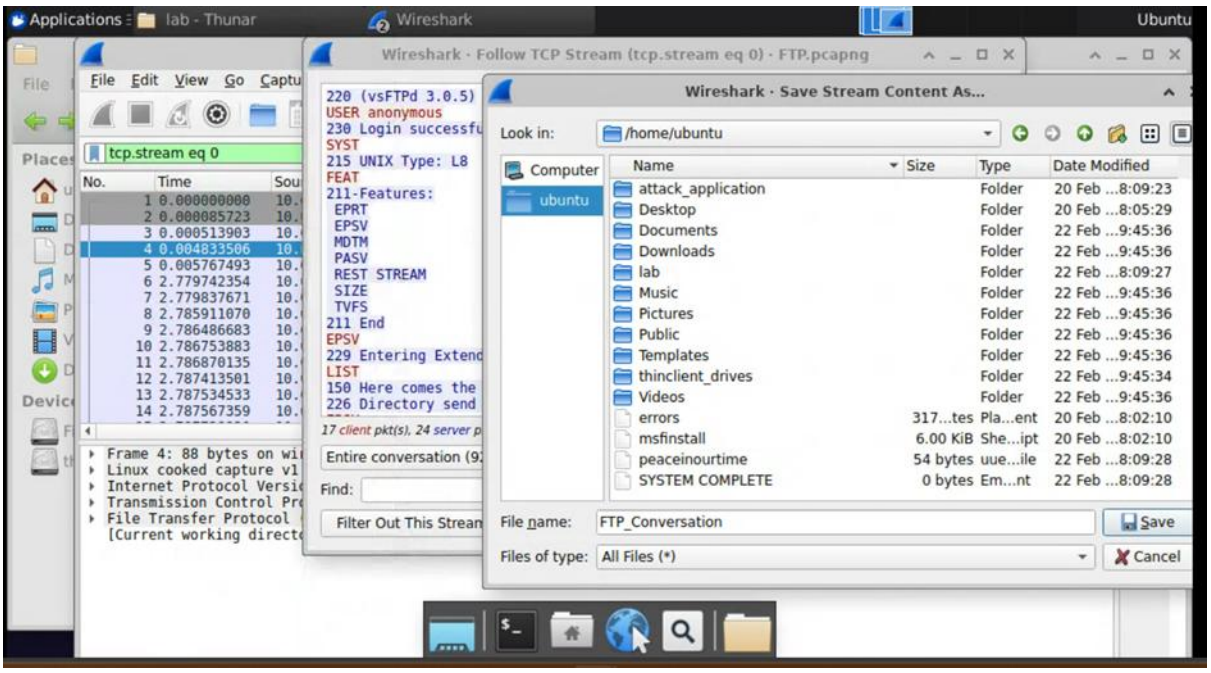

Figure 3.1 FTP traffic filtered, reconstructed using Follow TCP Stream, and exported as a file to preserve the full plaintext conversation for analysis.

Observation The FTP conversation was successfully exported for later analysis.

Why This Matters Exporting streams preserves evidence and allows further offline review during incident investigations or reporting.

Step 4: Locating the Login Event

Action Performed Selected "230 Login Successful" within the TCP stream and returned to the main Wireshark window.

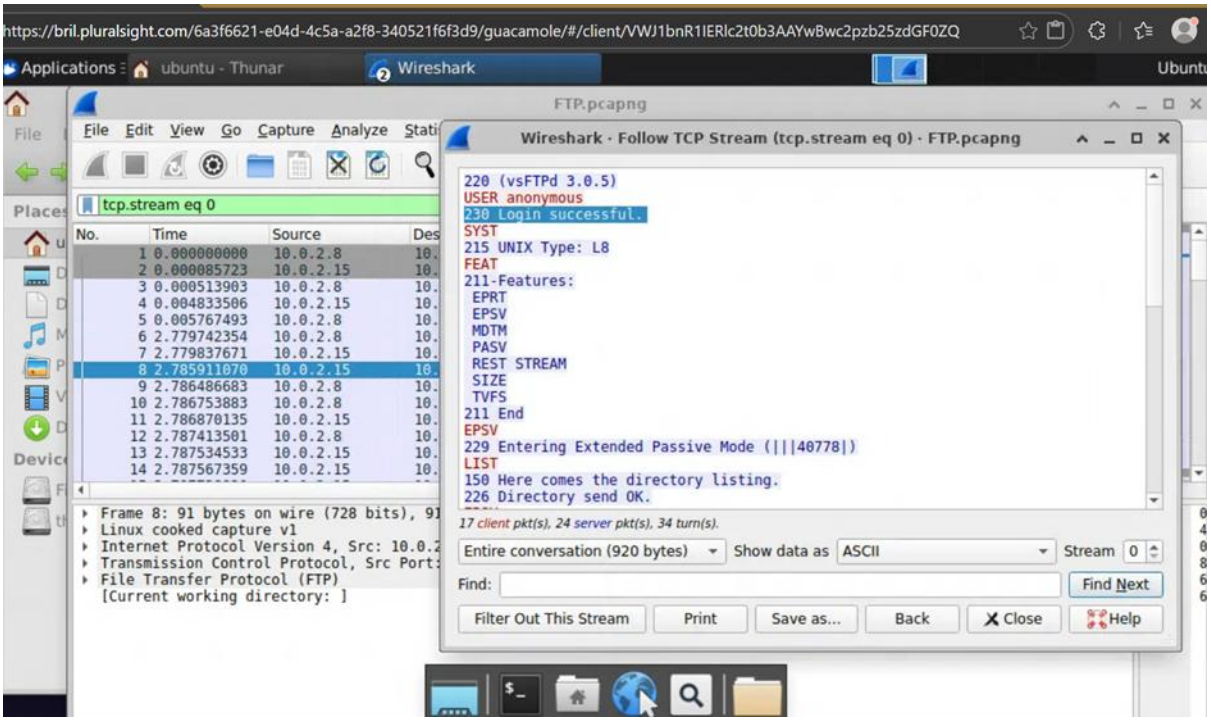

Figure 3.2 Follow TCP Stream view highlighting the FTP "230 Login Successful" event and the reconstructed plaintext conversation for session analysis.

Observation The corresponding packet was automatically highlighted in the packet capture.

Why This Matters This feature allows analysts to quickly locate specific events within long sessions, improving efficiency when analysing large packet captures.

<u>Step 5: Identifying the End of the FTP Session</u>

Action Performed Selected "221 Goodbye" in TCP stream and observed matching packet in main capture view.

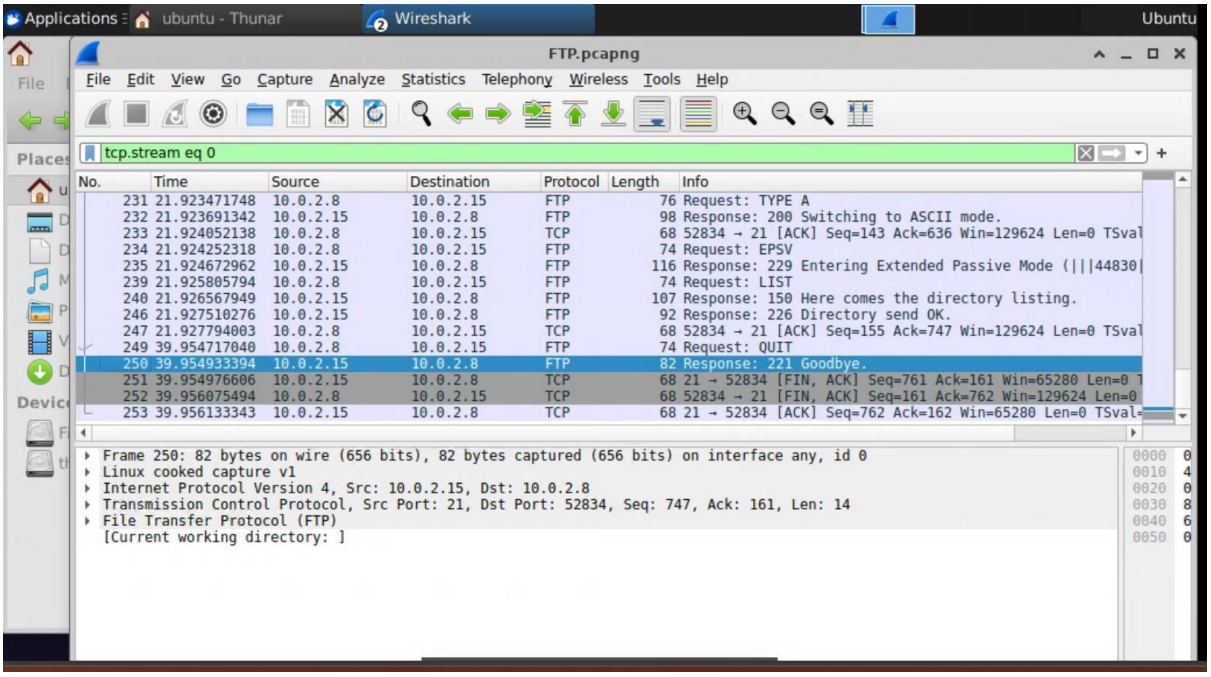

Figure 3.3 FTP session termination packet identified in the packet capture.

Observation The highlighted packet confirmed the closing event of the FTP session.

Why This Matters Identifying session start and end points helps analysts understand the timeline and scope of activity during network investigations.

Objective 4: Exporting Artifacts from Wireshark

Step 1: Identifying FTP Credentials in Traffic

Action Performed Reviewed the FTP packet capture and inspected packets containing authentication information to identify credentials transmitted during the session.

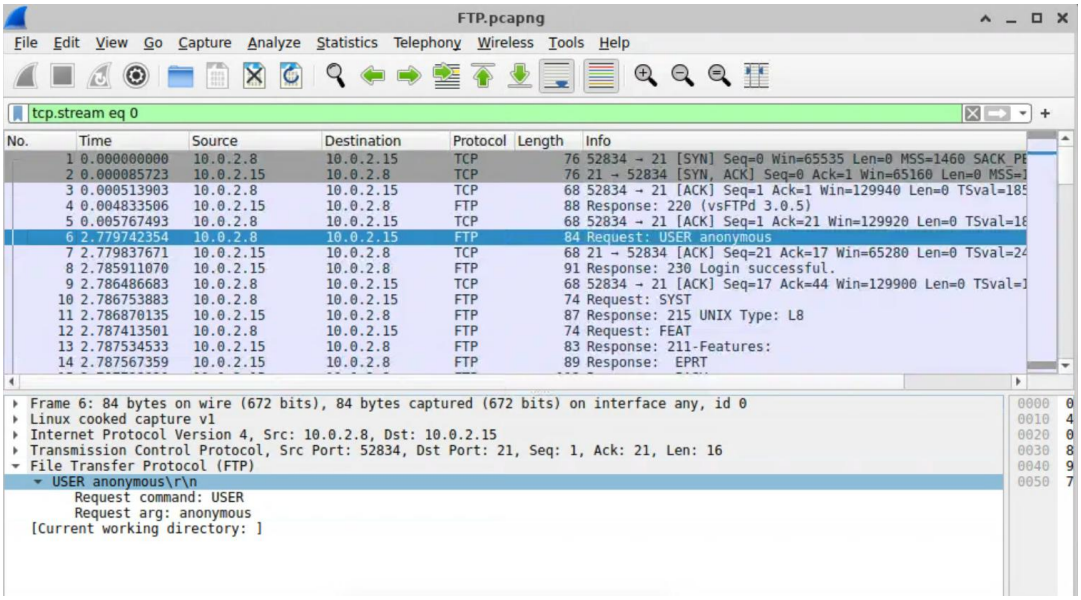

Figure 4.1 FTP username (USER anonymous) observed in plaintext within the packet capture.

Observation The FTP traffic showed authentication details, including the username, transmitted in plaintext within the packet capture.

Why This Matters Unencrypted protocols such as FTP expose sensitive information like usernames and passwords, making them easily recoverable if traffic is intercepted.

Step 2: Exporting File Artifacts from the FTP Stream

Action Performed Followed and exported the FTP stream to recover transferred files and reviewed the extracted content.

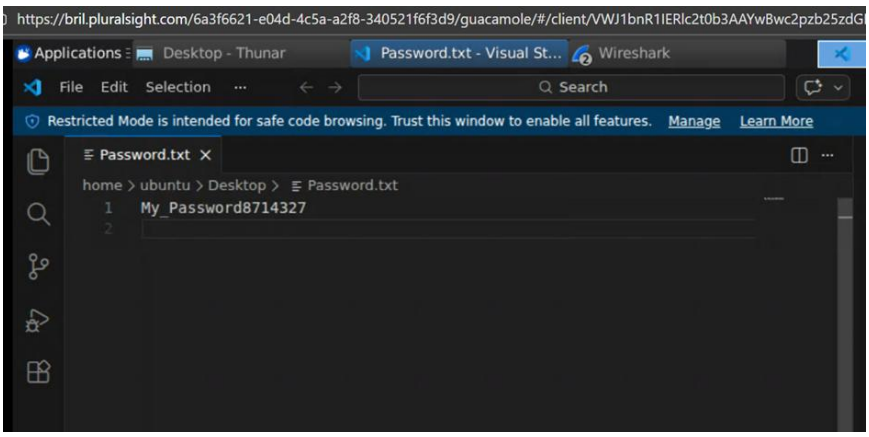

Figure 4.2 Exported FTP artifact showing recovered password saved to Password.txt.

Observation A file transferred during the FTP session was successfully exported and contained readable data, including a password value.

Why This Matters Exporting artifacts from packet captures demonstrates how attackers or analysts can reconstruct transferred files which is important for incident investigation and evidence collection.

Conclusion

This lab demonstrated how Wireshark can be used to analyze network traffic, identify protocol-level issues, and reconstruct application-layer activity. Through TCP analysis, retransmissions were identified and examined to understand potential causes of packet loss. ARP and DNS investigations highlighted how abnormal network behaviour can indicate spoofing or misconfigured resolution, while FTP stream analysis showed how unencrypted protocols expose sensitive information in plaintext.

Overall, the exercise reinforced the importance of packet-level analysis in troubleshooting and security investigations, as well as the risks associated with insecure protocols. The ability to filter traffic, follow TCP streams, and recover artifacts provides valuable skills for real-world network monitoring and incident response scenarios.